Measurement discipline and relevant information:

Measurement Discipline (System Configuration):

- System Specs:
- CPU: Intel Core i5 1135G7, 4c/8t, 400MHz-4.20GHz
- --> Turbo boost disabled via Intel P_State governor modifications
- RAM: 8GB LPDDR4X 4267MHz (Dual Channel)
- OS: Linux Debian Trixie 13.7.0
- Compiler/Tools:
- GCC 14.2.0 (Default with Debian 13)
- Governor: intel cpu_freq
- VSCode IDE
- FIO (version 3.39)

Environment Specific Setup:

- All CPU cores were locked to 1.40GHz constant
- Turbo boost disabled
- SMT/Hyperthreading enabled
- Idle/Sleep C states disabled (100% Active C0 state)
- No core/thread pinning was used

Project-specific:

- Partition size: 32GiB (32,768MiB)
- Filesystem: EXFAT
- SSD Model: Intel H10 32GB+512GB (Optane 32GB (PCIe 3.0 x2) + QLC 512GB (PCIe 3.0 x2), M.2 NVMe)
- --> In RAID (Intel RST) mode, but Debian only uses the 512GB QLC part, and doesn't use the optane caching at all. Hence the tests are only performed on the QLC portion of the drive.
- Methodology: Will be using filesystem instead of RAW due to it being non-destructive
- FIO Data file size: 12GB

Pre-experiment Hypothesis:

- Random reads will be significantly worse than sequential reads. Larger block sizes increase fragmentation, increase garbage collection, and slow down performance of the SSD. Small block sizes benefit from caching when sequential but not random.

How to use:

- Download these files:
- --> BenchmarkScript.sh
- --> BlockSizeSweep.fio

- --> MisleadingSetup.fio
- --> QueueDepthParallelismSweep.fio
- --> ReadWriteMixSweep.fio
- --> TailLatencyCharacterization.fio
- --> WorkingSetLBARangeDeviceStateNote.fio
- --> ZeroQueueBaselines.fio
- Note that these downloaded files must be in the same directory.
- Then, run the benchmark script with sudo priviledges with this command:
- --> sudo ./BenchmarkScript.sh

All-in-all, one run of this script should take:

- ~10-15 minutes
- ~100GiB Reads
- ~300GiB Writes

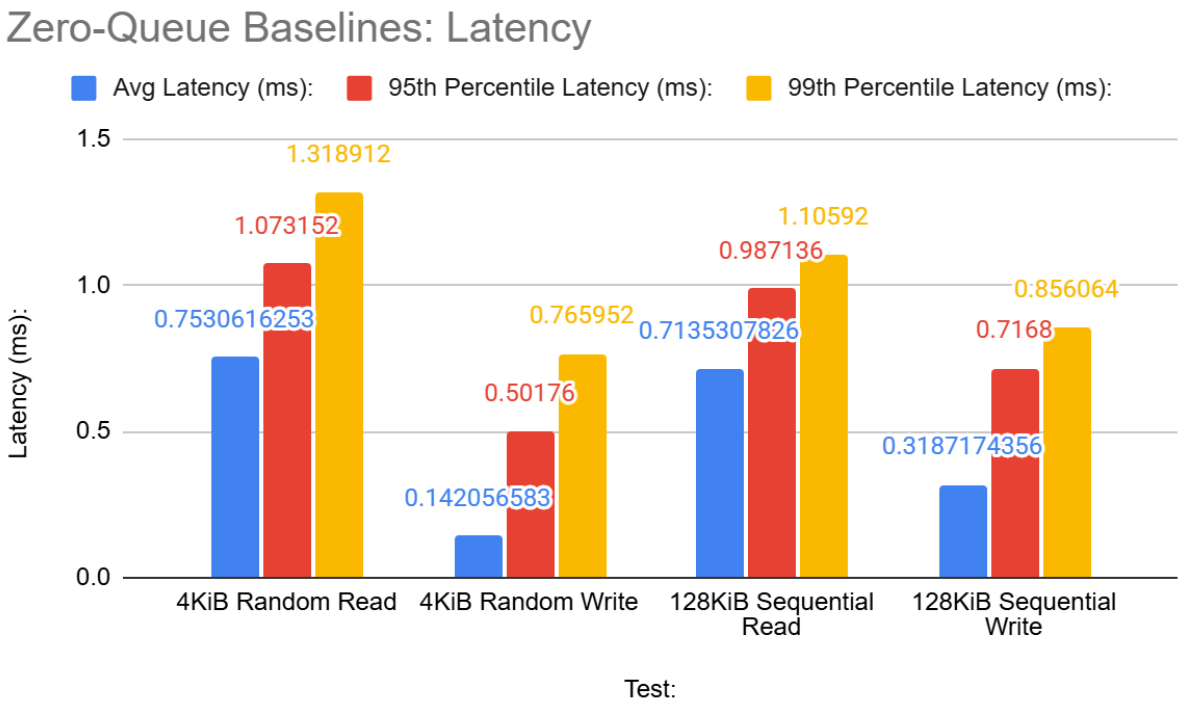
AI Use/Notes:

- Copilot was used to generate the FIO scripts. I double checked them and modified them to my liking
- --> The scripts were written by me, with assistance from Copilot and Gemini for documentation

Report + Required Experiments and Plots:

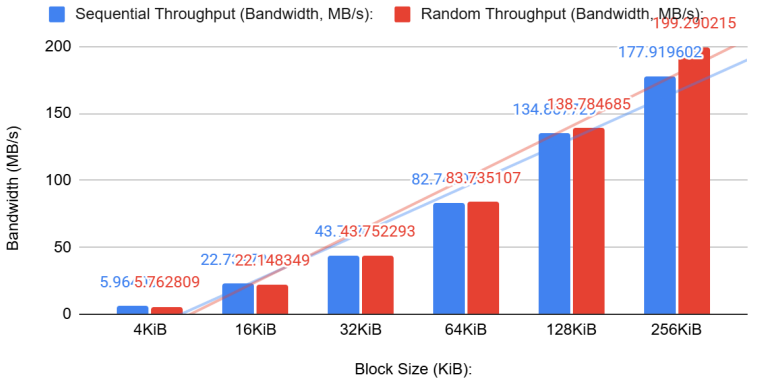
Pre-experiment hypothesis: Random reads will be significantly worse than sequential reads. Larger block sizes increase fragmentation, increase garbage collection, and slow down performance of the SSD. Small block sizes benefit from caching when sequential but not random.

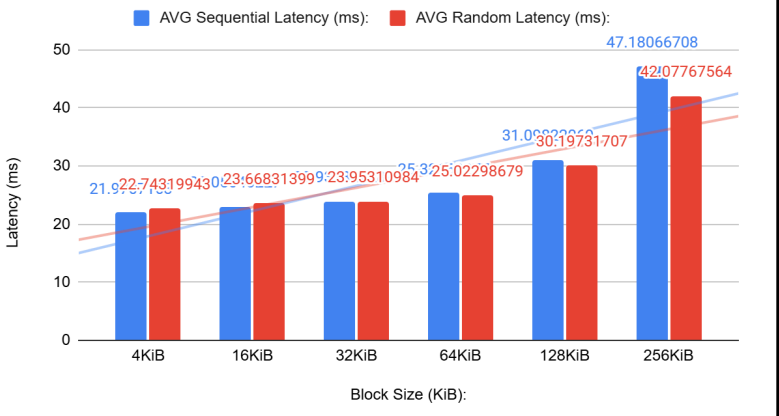
1. Zero-queue baselines:



4KiB random writes yielded the lowest latency, while 4KiB random reads yielded the highest latency. 128KiB sequential reads and writes fell in between, with reads having higher latency than writes. Read and write operations are not symmetric on this SSD, as writes are lower latency than reads.

2. Block-size sweep:



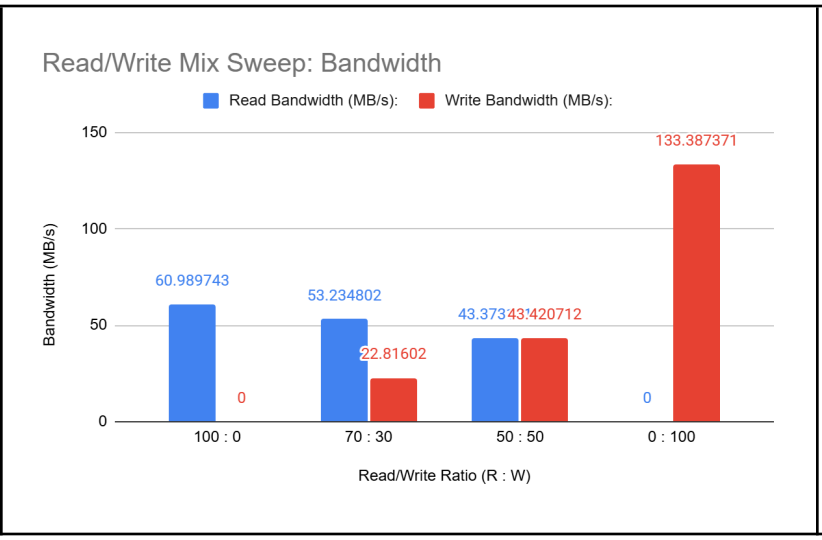


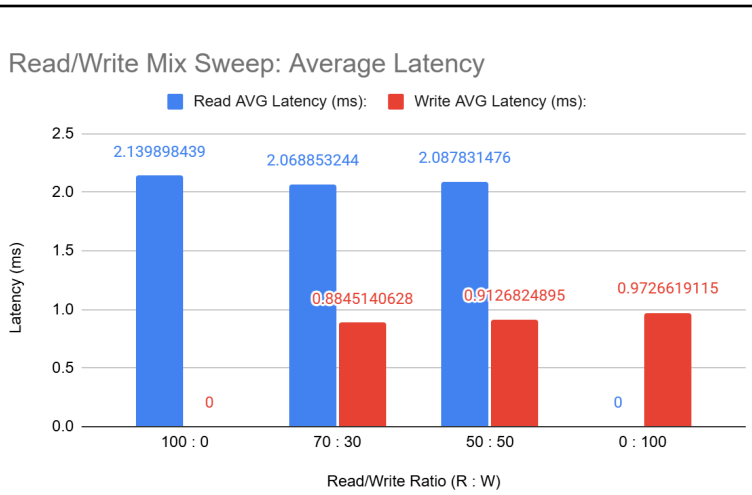
Overall, random and sequential access are pretty close to each other, with the differences only becoming more pronounced once block sizes hit 256KiB. Random read throughput gains the

lead over sequential throughput as block sizes increase, and grows at a faster rate. Random reads also have lower latency than sequential reads from working set sizes of 32KiB and higher.

Increasing the block sizes improves throughput because each operation moves more data at a time. But this also increases latency due to each memory transaction taking more time with larger blocks. This presents the tradeoff between bandwidth and latency in storage systems.

3. Read/write mix sweep:

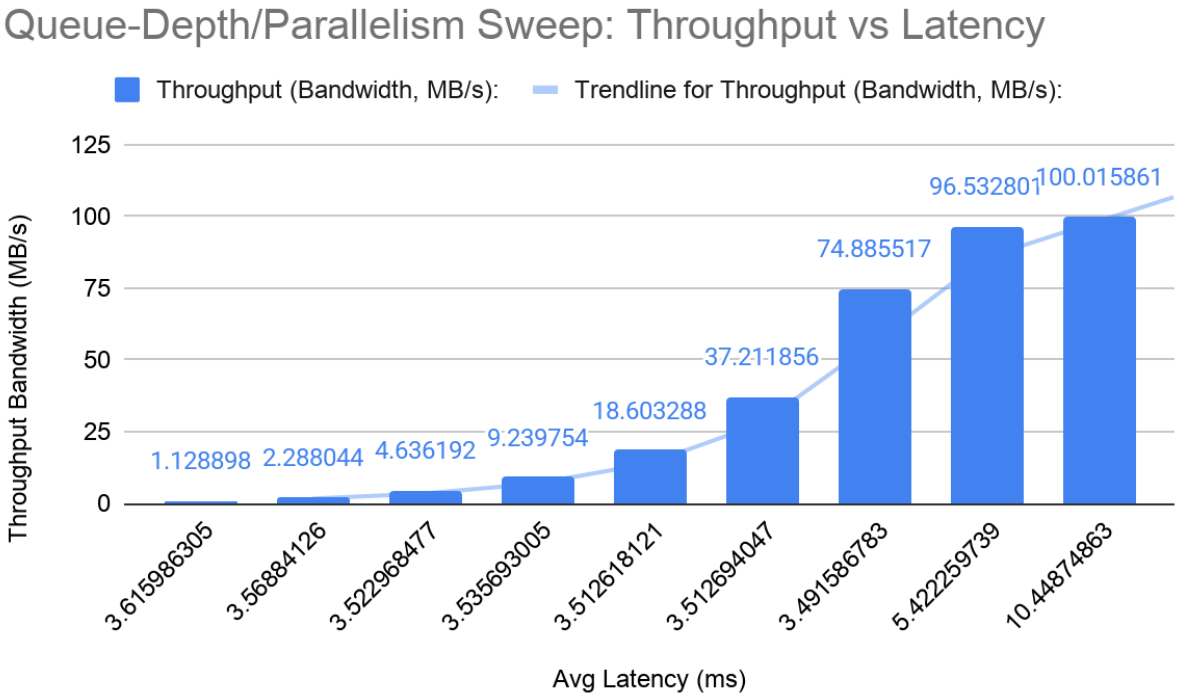




When it comes to bandwidth, reads and writes are asymmetric. 100% writes has over 2x the bandwidth of 100% reads. 50% read/write has a near-perfect split in bandwidth. Reads didn't drop very aggressively as the read ratio decreased. The write bandwidth increased dramatically as write ratios increased.

For latency, read latency barely changed across read/write mixes. Write latency also barely changed across read/write mixes. One thing to note is that write latency was less than half of the read latency.
Another interesting thing is that 100% reads had slightly higher latency than reads in the other mixes, and 100% writes had slightly higher latency than writes in other mixes.

4. Queue-depth/parallelism sweep:



It seems that the knee for average latency is around 5.4 milliseconds, as the throughput increases slow down from that point onwards, while the latency increases disproportionately.

For more context:

Queue Depth (QD):	Throughput (Bandwidth, MB/s):	Throughput (IOPS, optional):	AVG Latency (ms):
1	1.128898	275.609973	3.615986305
2	2.288044	558.58238	3.56884126
4	4.636192	1131.816364	3.522968477
8	9.239754	2255.643943	3.535693005
16	18.603288	4541.485201	3.512618121
32	37.211856	9084.236957	3.512694047
64	74.885517	18281.19723	3.491586783
128	96.532801	23564.75659	5.422259739
256	100.015861	24412.24698	10.44874863

Our latency stays relatively consistent until we hit QD=128 and QD=256. This means that we start hitting saturation around our knee at QD=128, at which throughput hits diminishing returns while latency skyrockets. QD=128 saturates our controller with requests.
Little's Law states that: L = B*W

- Where:
    - L = average number of outstanding requests (related to queue depth (QD))
    - B = throughput (bandwidth)
    - W = time / cycles (latency)
- As L increases with QD, B increases, and once it hits saturation with requests (L), then B barely increases, and W increases dramatically.

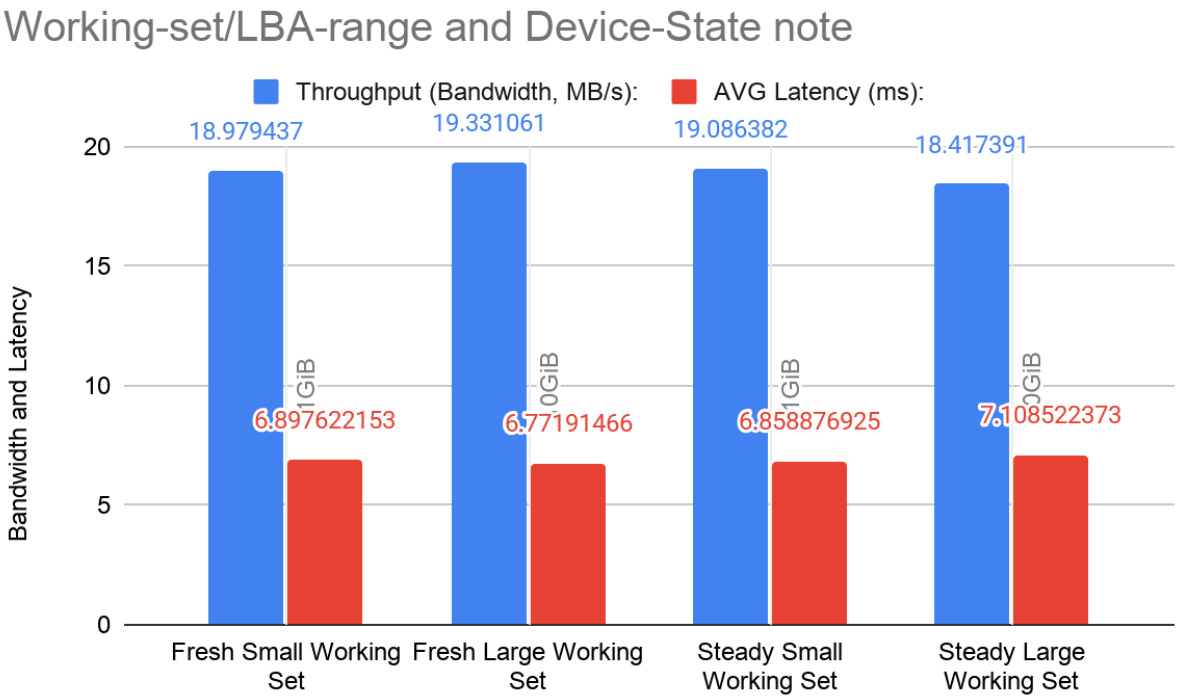
5. Tail-latency characterization:

I decided to simply use the json results from my QD parallelism sweep benchmark for this, as it is relevant.

Percentile:	Mid QD (=64):	Near-Knee QD (=128):
50	3.424256	5.341184
95	4.489216	6.848512
99	5.799936	8.716288
99.9	11.599872	12.255232

At every percentile, the near-knee QD yielded higher latency than the mid-QD. The near knee QD is when the SSD is put under load with IO requests, hence will have a higher latency than the mid-QD where it isn't fully saturated.

6. Working-set/LBA-range and device-state note:



There wasn’t a notable difference in throughput and latency when comparing fresh states to sustained steady states. For the large working set, there was a slight increase in latency and a minor drop in throughput. For the small working set, the difference was even smaller, with a slight decrease in latency, and slight decrease in throughput.

Small working set size = 1 GiB
Large working set size = 10 GiB

7. Misleading setup:

My original FIO jobfile for the working-set and device-state experiment contained a misleading setup. I initially assumed that individual job sections could be treated independently, so I only defined the steady-state benchmarking sections. However, FIO executes the sections in the order they appear within the jobfile. As a result, the preconditioning workload ran before both of the working-set measurements, causing the supposed "fresh-state" measurements to actually be collected from a preconditioned SSD. This produced misleading results because the fresh-state and steady-state measurements were no longer comparable. The issue was corrected by adding separate fresh-state job sections before the preconditioning phase and then rerunning the experiment.